UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA

REPORTE DE PRÁCTICA Nº 10

NOMBRE COMPLETO: Gómez Aceves Abraham Agustín
Nº de Cuenta: 320093290
GRUPO DE LABORATORIO: 03
GRUPO DE TEORÍA: 05
SEMESTRE: 2027-1
FECHA DE ENTREGA LÍMITE: 07 – Noviembre – 2026
CALIFICACIÓN: _____

Índice

1. Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa. 2

1.1. Secciones de código que modifiqué 2

1.2. Capturas de pantalla de la ejecución 4

2. Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue 5

3. Conclusión 5

4. Bibliografía 6

1. Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Descripción del ejercicio

Práctica 10 (Rama `Cohete_keyframes`): Animación Compleja por Keyframes. Se pide que el cohete despegue, se separe en fases (módulos) y que, al final, la cápsula de la punta quede orbitando mientras las demás piezas caen al piso. A diferencia de la versión procedural, esta rama utiliza un sistema de **keyframes** guardados y leídos desde un archivo de texto (`.txt`) para controlar la posición, rotación y escala de cada pieza a lo largo del tiempo.

1.1 Secciones de código que modifiqué

1. Estructura, lectura y guardado de Keyframes (`KeyframesCohete.h`)

Se creó la clase `AnimacionCohete` que se encarga de leer el archivo de texto (`keyframes_cohete.tx` interpolar los datos linealmente (LERP) y guardar el estado actual de las 5 piezas del cohete. Además, incluye funciones para imprimir en consola y guardar los keyframes en un nuevo archivo.

```
// Funcion para cargar el archivo de texto con los keyframes
bool Carga(const char* ruta)
{
    std::ifstream archivo(ruta);
    if (!archivo.is_open())
    {
        printf("[keyframes] no se pudo abrir %s\n", ruta);
        return false;
    }
    // ... (lectura de lineas y parseo)
    printf("[keyframes] %d keyframes cargados de %s (duracion %.2f s)\
   n", (int)frames.size(), ruta, frames.back().tiempo);
    return true;
}

// Funcion de interpolacion lineal (LERP) entre keyframes
void Interpola()
{
    int n = Segmento();
    const KeyframeCohete& a = frames[n];
    const KeyframeCohete& b = frames[n + 1];
    float u = (tiempo - a.tiempo) / (b.tiempo - a.tiempo);

    fuego = a.fuego + (b.fuego - a.fuego) * u;
    piezaFuego = a.piezaFuego;
    for (int i = 0; i < KF_PIEZAS; i++)
    {
        pos[i] = a.pos[i] + (b.pos[i] - a.pos[i]) * u;
        rot[i] = a.rot[i] + (b.rot[i] - a.rot[i]) * u;
    }
}
```

2. Captura de teclado para controlar la animación (`Window.cpp`)

En la clase `Window` se agregaron las teclas para controlar la animación por keyframes: **B** para reproducir, **R** para detener, **I** para imprimir los keyframes en consola y **O** para guardarlos en un archivo de texto.

```
// La B reproduce la animacion por keyframes
if (key == GLFW_KEY_B && action == GLFW_PRESS)
{
    theWindow->zonaPlaneta = 2; // Mueve la camara al planeta 2
    if (theWindow->modoCamara != 2)
        theWindow->modoCamara = 1;
    theWindow->iniciosKeyframes += 1;
}
// La R detiene y regresa el cohete a la plataforma
if (key == GLFW_KEY_R && action == GLFW_PRESS)
{
    theWindow->reinicioKeyframes += 1;
}
// La I imprime los keyframes cargados en consola
if (key == GLFW_KEY_I && action == GLFW_PRESS)
{
    theWindow->imprimeKeyframes += 1;
}
// La O guarda los keyframes actuales en un txt
if (key == GLFW_KEY_O && action == GLFW_PRESS)
{
    theWindow->guardaKeyframes += 1;
}
```

3. Actualización y dibujo por keyframes (`main.cpp`)

En el ciclo principal, se actualiza el tiempo de la animación usando la variable `deltaTime` (dividida entre 61 para convertirla a segundos reales). Posteriormente, se dibuja cada pieza del cohete aplicando su propia matriz de traslación y rotación calculada por el keyframe.

```
// Actualiza el tiempo de la animacion por keyframes
animCohete.Actualiza(deltaTime / 61.0f);

// Funcion que dibuja el cohete con los datos del keyframe actual
void DibujaCoheteKeyframes(Model piezas[], AnimacionCohete& anim, glm
    ::mat4 raiz, GLuint uniformModel, glm::mat4 matrices[])
{
    for (int i = 0; i < NUM_PIEZAS_COHETE; i++)
    {
        glm::mat4 model = raiz;
        model = glm::translate(model, centrosCohete[i] + anim.pos[i]);
        model = glm::rotate(model, anim.rot[i].x * toRadians, glm::
    vec3(1.0f, 0.0f, 0.0f));
        model = glm::rotate(model, anim.rot[i].y * toRadians, glm::
    vec3(0.0f, 1.0f, 0.0f));
        model = glm::rotate(model, anim.rot[i].z * toRadians, glm::
    vec3(0.0f, 0.0f, 1.0f));
        model = glm::translate(model, -centrosCohete[i]);
        matrices[i] = model;

```

```
17          glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(
    model));
18          piezas[i].RenderModel();
19      }
20  }
```

1.2 Capturas de pantalla de la ejecución

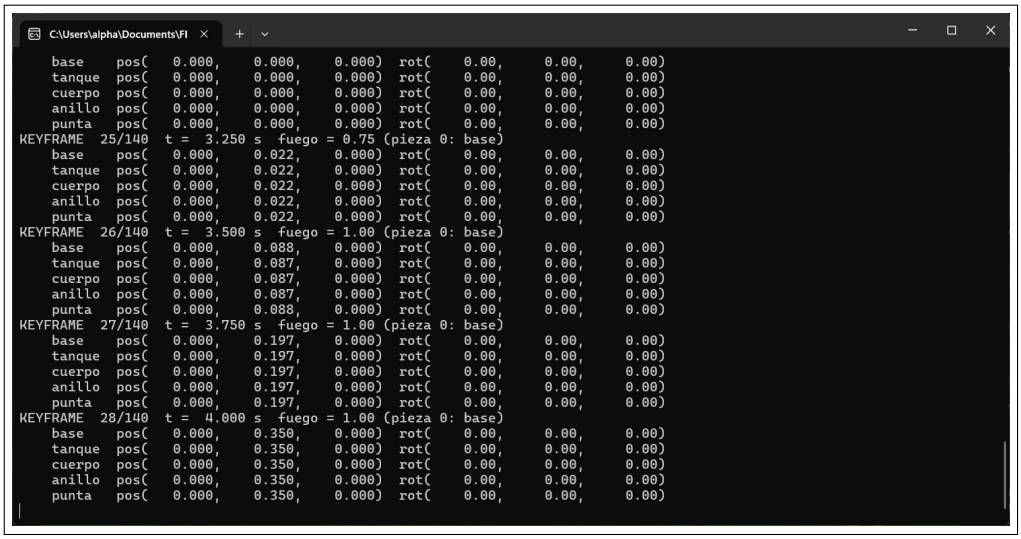


Figura 1: Captura 1: Impresión en consola / archivo `.txt` mostrando los keyframes guardados (tiempo, posición y rotación de cada pieza) y el momento exacto de la separación del cohete.

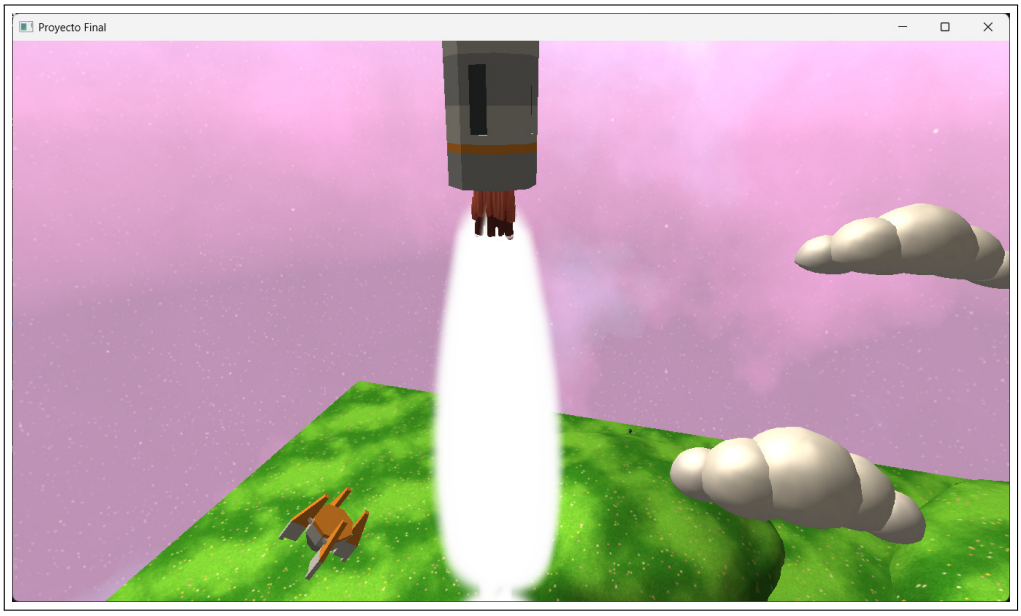


Figura 2: Captura 2: Ejecución del programa. Se observa la cápsula de la punta orbitando en el espacio, mientras las demás piezas (cuerpo y propulsores) caen al piso tras la separación.

2. Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue

1. **Sincronización de la separación con los keyframes:** Al principio, las piezas se separaban bruscamente porque los tiempos en el archivo `.txt` no coincidían con el ciclo de renderizado. *Solución:* Se implementó un sistema de interpolación lineal (LERP) entre el keyframe anterior y el siguiente, asegurando que la transición de separación fuera suave y basada en el tiempo real (`dtSeg = deltaTime / 61.0f`).

2. **Jerarquía de las piezas al caer:** Las piezas caían al piso, pero la cápsula seguía heredando la matriz de transformación del cuerpo principal, por lo que se movía junto con él. *Solución:* Se modificó la estructura para que, al llegar al keyframe de separación, cada pieza use su propia matriz de órbita o caída independiente, calculada directamente desde el archivo de texto.

3. **Pérdida de precisión en el archivo .txt:** Al guardar las posiciones con pocos decimales, el cohete daba pequeños "saltos" al reproducir la animación. *Solución:* En la función `Guarda()` de `KeyframesCohete.h` se aseguró el uso de suficiente precisión para las posiciones (X, Y, Z) y rotaciones, garantizando una reproducción idéntica a la original.

3. Conclusión

a. Los ejercicios del reporte: Complejidad, Explicación

La complejidad de esta práctica radicó en el cambio de paradigma: pasar de una animación procedural (basada en funciones matemáticas en tiempo real) a una animación basada en **keyframes** (datos precalculados). El mayor reto fue diseñar la estructura de datos para el archivo `.txt`, programar el lector del mismo y aplicar correctamente las matrices de transformación individuales para cada una de las 5 piezas del cohete.

b. Comentarios generales

El uso de archivos externos para las animaciones es una técnica muy poderosa, ya que permite modificar la animación sin necesidad de recompilar el código. Sería bueno que se mostrara cómo exportar animaciones desde Blender a un formato de texto para consumirlo en OpenGL.

c. Conclusión

La práctica demostró que los keyframes ofrecen un control absoluto sobre el resultado final, permitiendo animaciones más complejas y realistas (como la separación de fases y la caída de escombros). Si bien consumen más memoria que las ecuaciones matemáticas, su versatilidad para reproducir cinemáticas complejas los convierte en una herramienta esencial en el desarrollo de proyectos finales de computación gráfica.

4. Bibliografía

- Angel, E., & Shreiner, D. (2012). *Interactive computer graphics: A top-down approach with shader-based OpenGL* (6a ed.). Addison-Wesley.
- Farin, G. (2002). *Curves and surfaces for CAGD: A practical guide* (5a ed.). Morgan Kaufmann.
- G-Truc Creation. (s.f.). *OpenGL Mathematics (GLM) manual*. `https://github.com/g-truc/glm/blob/master/manual.md`
- Hearn, D., Baker, M. P., & Carithers, W. (2011). *Computer graphics with OpenGL* (4a ed.). Pearson.
- Khronos Group. (2024). *OpenGL 4.6 API core profile specification*. `https://www.khronos.org/opengl/`